SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA5103 – Cryptography and Network Security

ASSIGNMENT 2 – CO2

Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.

Student Name	Register Number	Department / Section	Academic Year / Semester	Faculty Name	Nature	Duration	Total Marks	Date of Submission	Date of Evaluation
G. Yashmita	192511032	CSE	2026	Dr.R.Dharani	Individual	1 week	50	23.10.26	24.10.26

GITHUB LINK: https://github.com/192511032-YashmitaG/CSA5103-Cryptography-C.git

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks / Evidence Expected
1	Symmetric Cryptosystem Selection and Modes of Operation	7		Explain DES, 3DES, Blowfish and RC5 structure, block/key characteristics and rounds; compare block and stream ciphers; analyse ECB, CBC, CFB, OFB and CTR, including IV/nonce use, leakage, error propagation and parallelism. Implement/simulate at least three algorithms and provide pattern/IV demonstrations and performance results.
2	Number-Theory Engine and Prime Selection for RSA	6		Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation. Explain Euler's totient and gcd(e, φ(n)) = 1; compare trial division, Fermat and Miller–Rabin using toy candidates, including Carmichael numbers; record operation counts and discuss prime density and larger RSA sizes.
3	RSA Encryption/Decryption, Weaknesses and Padding	7		Generate RSA keys and implement encryption/decryption; verify a hand-checkable example; test deterministic/repeated messages, small-exponent behaviour, boundary inputs and invalid parameters. Demonstrate toy factorisation attacks; explain weak key generation, modulus limitations and OAEP padding.
4	Diffie–Hellman, Authentication and Key Management	7		Derive and implement Diffie–Hellman; show private/public values and shared-key computation; test toy examples and measure brute-force discrete-log effort. Demonstrate unauthenticated MITM and explain signatures/certificates. Propose key generation, storage, rotation, revocation, recovery and session-key lifetime policies.
5	ECC and Public-Key Strategy Comparison	5		Explain elliptic-curve public-key mathematics and demonstrate toy ECC key generation/agreement. Compare RSA, DH and ECC by key size, computation, bandwidth and deployment; discuss secure randomness, parameter validation and private-key protection; defend a strategy for constrained devices.
6	Digital Signatures, Hybrid Encryption and Quantum Readiness	7		Design hash-then-sign and signature verification; compare RSA, DSA/DSS and ECDSA. Design authenticated hybrid encryption for large files; test tampering of files, signatures and session keys; compare direct RSA and hybrid operation/file-size costs. Explain QKD, quantum threats to RSA/ECC and migration readiness.
7	Integrated Design Challenge: Diagnose, Redesign and Validate	11		A: Diagnose a bottleneck/weakness using earlier evidence (2). B: State a measurable redesign objective (1). C: Design a revised cryptographic architecture/protocol and explain properties, reduced operations and trade-offs (3). D: Integrate quantum readiness (1). E: Predict, implement and validate the redesign using comparable evidence (2). F: Defend the final design, residual risks and deployment trade-offs (2).
	TOTAL	**50**		**Marks to be awarded by the faculty.**

SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

OVERALL REMARKS

Faculty feedback:

Student Signature	Faculty Signature	Program Director
		

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present P = partly present ✗ = missing. Mark the status after checking the student's submitted report and supporting evidence.

Question	Evidence Expected	Status	Note
1	Algorithm/cipher comparison; mode comparison; three-algorithm test evidence; ECB pattern and IV/nonce demonstration; performance table.	☐	Evidence should be traceable to the submitted report, code or experimental output.
2	Mathematical derivations and pseudocode; designed test cases; modular operation counts; primality-test results including Carmichael numbers; engineering decision.	☐	Evidence should be traceable to the submitted report, code or experimental output.
3	RSA implementation and hand-worked example; prediction-versus-result table; attack/factorisation traces; padding analysis; security conclusions.	☐	Evidence should be traceable to the submitted report, code or experimental output.
4	DH derivation and message-sequence diagram; implementation output; MITM trace; authentication explanation; growth measurements; key-management policy.	☐	Evidence should be traceable to the submitted report, code or experimental output.
5	Toy ECC calculation; RSA/DH/ECC comparison table; constrained-device recommendation supported by Q2–Q4 evidence.	☐	Evidence should be traceable to the submitted report, code or experimental output.
6	Signature flow; hybrid-encryption design; tamper-test results; direct-RSA/hybrid comparison; quantum-readiness explanation; final architecture.	☐	Evidence should be traceable to the submitted report, code or experimental output.
7	Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence, validation and engineering defence.	☐	Evidence should be traceable to the submitted report, code or experimental output.

IMPLEMENTATION EVIDENCE CHAIN(Tasks 5–7)

Evidence Item	Status	Observation
Source code	☐	Readable student-written core algorithms; language, dependencies and execution instructions stated.
Test inputs and keys	☐	Student-created plaintexts, keys, toy parameters, edge cases and test data recorded.
Expected behaviour	☐	Predicted ciphertexts, shared keys, signatures, attack outcomes and recovered plaintext documented.
Actual output	☐	Program output, screenshots or logs included and reproducible.
Intermediate calculations	☐	Modular arithmetic, key-generation steps, protocol values and transformations shown.
Attack evidence	☐	Toy-only attack traces for MITM, factoring, weak parameters, pattern leakage or tampering where required.
Performance measurements	☐	Operation counts, loop/division counts, execution time and input/key-size conditions recorded.

Evidence Item	Status	Observation
Boundary/failure tests	☐	Invalid parameters, boundary messages, repeated inputs, tampering and relevant failure cases considered.
Interpretation	☐	Results connected to mathematical properties, security assumptions and deployment constraints.
Consistency	☐	Code, raw data, tables, graphs, diagrams and report claims agree.

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All questions Q1–Q7 and Q7 parts A–F answered	☐ Yes ☐ Partly ☐ No
All required mathematical derivations and pseudocode included	☐ Yes ☐ Partly ☐ No
Core algorithms implemented by the student; libraries used only for cross-checking	☐ Yes ☐ Partly ☐ No
Student-created toy parameters and test data used for all attack demonstrations	☐ Yes ☐ Partly ☐ No
Raw experimental data, operation counts and execution-time conditions provided	☐ Yes ☐ Partly ☐ No
Tables/graphs and protocol or message-sequence diagrams included	☐ Yes ☐ Partly ☐ No
Predictions compared with actual experimental evidence	☐ Yes ☐ Partly ☐ No
Security limitations, trade-offs and deployment decisions justified	☐ Yes ☐ Partly ☐ No
Source code and reproducible execution procedure provided	☐ Yes ☐ Partly ☐ No
Report numbering, cross-references and figures are consistent	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders remain	☐ Yes ☐ Partly ☐ No
Student can explain and defend the work during verification/viva	☐ Yes ☐ Partly ☐ No

Safety note: All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

Assessment note: A brief verification/viva may assess algorithmic and protocol design, source code, test-data/key-set design, predictions, experimental results, security analysis and the Q7 redesign.

SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 2

QUESTIONS

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.
CO2 Coverage: Symmetric key cryptosystems; Data Encryption Standard (DES); Triple DES (3DES); Blowfish; RC5; block ciphers; stream ciphers; block cipher modes of operation; public-key cryptosystems; RSA algorithm; key distribution and management; Diffie–Hellman key exchange; elliptic curve cryptography (ECC); introduction to quantum cryptography; number-theory foundations; RSA attacks and padding; digital signatures; hybrid encryption.

Assignment Title: Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network (Total Marks: 50)

Scenario — Secure Healthcare Data Exchange
A healthcare network connects one central hospital with remote clinics that exchange patient records, prescriptions, diagnostic reports and billing data over public networks. The network contains low-resource devices, high-volume file transfers and multiple security requirements. The security team must select appropriate symmetric and public-key techniques, establish keys securely, authenticate exchanges and prepare for long-term cryptographic migration. The system must:

- Protect stored and transmitted clinical records with confidentiality.
- Establish session keys over an observable public network.
- Provide authentication and integrity for security-sensitive exchanges.
- Support large files without performing expensive public-key encryption on every data block.
- Remain practical for constrained clinics while allowing secure migration as threats and technology change.

Design Challenge
During peak operation the network expands to many remote sites. An attacker can observe traffic, attempt a man-in-the-middle attack during unauthenticated key exchange, exploit weak RSA parameters, and analyse repeated ciphertext patterns. Management also requires a migration plan for future quantum threats. Re-evaluate the original design using your mathematical and experimental evidence.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)
Problem Context – Protecting Clinical Records
The network must compare classical symmetric choices before using public-key techniques. Evaluate DES, 3DES, Blowfish and RC5, together with the distinction between block and stream ciphers and the principal block-cipher modes.
Your Challenge

- Explain the structure, block size, key characteristics and round operation of DES, 3DES, Blowfish and RC5.
- Implement or simulate at least three of these algorithms on student-created test data and verify encryption/decryption.
- Explain the difference between block and stream ciphers and identify an appropriate use for each.
- Explain ECB, CBC, CFB, OFB and CTR, including IV/nonce requirements, pattern leakage, error propagation and parallelism.
- Demonstrate why ECB can reveal repeated plaintext patterns and why IV/nonce reuse is dangerous.
- Compare operation count and execution time for increasing message sizes.

Evaluation

Rank DES, 3DES, Blowfish and RC5 for the scenario, identify legacy limitations, and recommend where a block or stream construction would be appropriate. Do not treat a larger key space alone as proof of security.
Evidence Expected: Algorithm comparison, mode comparison, test results, pattern/IV demonstration and a justified recommendation.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)
Problem Context – RSA Key Generation
RSA depends on modular arithmetic, modular inverses, modular exponentiation and the generation of suitable primes.
Your Challenge

- Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation.
- Compare trial-search inverse with Extended Euclid and naive repeated multiplication with square-and-multiply.
- Explain Euler's totient function and the condition gcd(e, φ(n)) = 1.
- Compare trial division, Fermat and Miller–Rabin on safe toy candidates, including Carmichael numbers such as 561, 1105 and 1729.
- Record modular multiplications, division/loop steps and false acceptance results.
- Explain how prime density affects the number of candidates tested.
- Re-evaluate the methods when the RSA modulus grows from classroom-sized values toward 1024-, 2048- and 3072-bit engineering requirements.

Evidence Expected: Mathematical derivations, pseudocode, designed test cases, operation counts, primality-test evidence and engineering decision.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)
Problem Context – Online Banking Message Protection
A bank uses textbook RSA for short tokens. The security team must determine whether correct decryption is sufficient evidence of secure encryption.
Your Challenge

- Generate RSA keys and implement C = M^e mod n and M = C^d mod n.
- Verify one hand-checkable RSA example.
- Test deterministic encryption, repeated-message encryption, small-message/small-exponent behaviour, boundary messages and invalid parameter choices.
- Explain how factoring n permits recovery of φ(n) and d.
- Implement or simulate trial-division and Fermat factorisation on weak toy moduli, including close primes and a small factor.
- Explain why RSA key length alone does not guarantee security and why weak key generation can undermine a large modulus.
- Explain RSA padding, especially OAEP, and how randomisation prevents structural weaknesses of textbook RSA.

Inquiry
Predict each test before execution, compare prediction with evidence, and state the limitations of classroom-scale factoring experiments.
Evidence Expected: Correct RSA implementation, attack traces, factoring evidence, padding analysis and security conclusions.
Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)
Problem Context – Secure Link Between Remote Clinics
Two clinics must establish a shared session key over a network that an attacker can fully observe, without having a pre-shared secret.
Your Challenge

- Derive the Diffie–Hellman procedure from (g^a)^b ≡ (g^b)^a (mod p).
- Develop pseudocode showing private values, public values and the shared-key computation.
- Test a small hand-verifiable example and a larger toy configuration.
- Measure the legitimate computation and a brute-force discrete-log attack as p grows.
- Demonstrate an active man-in-the-middle attack against unauthenticated Diffie–Hellman.
- Explain how digital signatures or certificates authenticate the exchanged values and defeat the attack.
- Design a key-distribution and key-management policy covering generation, storage, rotation, revocation, compromise recovery and session-key lifetime.

Evidence Expected: Protocol derivation, message-sequence diagram, implementation, MITM demonstration, growth analysis and key-management policy.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Problem Context – Constrained Devices

Remote medical devices need public-key security with manageable key size, computation and communication overhead.

Your Challenge

- Explain the mathematical basis of elliptic-curve public-key cryptography and demonstrate a toy ECC key-generation/key-agreement example.
- Compare RSA, Diffie–Hellman and ECC in key size, computational cost, bandwidth and deployment considerations.
- Explain why ECC can provide comparable security with smaller public-key material than traditional RSA-style parameters.
- Identify implementation requirements such as secure randomness, parameter validation and protected private keys.
- Recommend a public-key strategy for constrained devices and justify it using the experimental evidence from Q2–Q4.

Evidence Expected: Toy ECC example, RSA/DH/ECC comparison table and a defended recommendation.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

Problem Context – Large Confidential Document Exchange

The healthcare network must transfer large files with confidentiality, integrity and sender authenticity. The team is comparing direct public-key protection with hybrid encryption and must also consider future quantum threats.

Your Challenge

- Design a digital-signature process using hash-then-sign and explain signing and verification.
- Compare RSA, DSA/DSS and ECDSA at the conceptual level with respect to keys, signing/verification, randomness and security assumptions.
- Design hybrid encryption: encrypt the large payload with a random symmetric session key, then protect or establish that session key using RSA or Diffie–Hellman; authenticate the exchange.
- Test tampering of the file, signature and protected session key.
- Compare direct RSA encryption of large data with hybrid encryption using operation counts and file-size growth.
- Explain the basic principle of quantum key distribution/quantum cryptography and how eavesdropping can be detected.
- Explain why sufficiently capable quantum computers threaten RSA and ECC and propose a migration/readiness strategy without claiming that quantum cryptography itself replaces ordinary cryptographic key management.

Deployment Decision

Choose a practical architecture for the healthcare network and justify the choice using performance, security, key-management burden and long-term risk.

Evidence Expected: Signature flow, hybrid-encryption design, tamper tests, growth comparison, quantum-security explanation and final architecture.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Review the evidence from Q1–Q6 and identify one significant computational or security limitation that becomes important as message volume, key size, number of clinics or attacker capability increases.

A. Diagnose the Bottleneck or Weakness (2 Marks)

- Identify the operation or missing security property responsible for the limitation.
- Explain how its cost or risk changes with input/key size or repeated use.
- Use measured operation counts or attack evidence from earlier tasks.

B. Define the Design Objective (1 Mark)

State a measurable redesign objective, such as reducing public-key operations, preventing a man-in-the-middle, reducing key-management exposure, detecting weak keys, or improving secure bulk-data handling.

C. Design the Revised Strategy (3 Marks)

Develop a revised architecture using the appropriate combination of symmetric encryption, secure modes, RSA/DH/ECC, digital signatures, key management and hybrid encryption. Explain the mathematical/security property exploited, operations reduced, additional state or keys required, and new trade-offs.

D. Integrate Quantum Readiness (1 Mark)

Identify which RSA/ECC dependencies are threatened by future quantum computing and define a migration approach. Explain separately what quantum key distribution can contribute and what it cannot replace.

E. Predict, Implement and Validate (2 Marks)

Predict the improvement before implementation. Compare original and redesigned solutions using operation counts, attack steps, key-management events, verification results and execution time where useful. Verify that the redesign preserves required functionality.

F. Defend the Final Design (2 Marks)

Defend the final decision against the original alternatives. Discuss residual risks, deployment cost, usability, key compromise, MITM attacks, weak keys, large-file performance and long-term quantum risk.

Evidence Expected: Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence and final engineering defence.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/RC5, block/stream ciphers and modes	7
Q2	Number theory, modular arithmetic and prime selection	6
Q3	RSA encryption/decryption, attacks, weak keys and padding	7
Q4	Diffie–Hellman, MITM authentication and key management	7
Q5	ECC and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid encryption and quantum readiness	7
Q7	Day-4 integrated diagnose, redesign and validation	11
TOTAL		50

Submission Requirements

Submit your assignment report containing Q1–Q7 evidence, source code for student-implemented core algorithms, raw experimental data, tables/graphs and protocol/message-sequence diagrams. Libraries may be used for cross-checking standard primitives, but the mathematical core and experiments must be your own.

All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

Assessment Verification

Students may be asked to explain algorithm design, source code, test data, mathematical predictions, experimental results, security analysis and the Day-4 redesign during a brief verification/viva. Both the reasoning and the evidence must be understood by the student.

- Algorithmic and protocol design
- Source code
- Test-data and key-set design
- Experimental results
- Predictions
- Security analysis
- Design Challenge

during a brief verification discussion/viva. The student should therefore be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook cryptosystems.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

ABSTRACT

The increasing adoption of interconnected healthcare systems has created a significant requirement for secure communication and information exchange between hospitals, remote clinics and centralised medical databases. Healthcare organisations transmit sensitive information, including patient records, diagnostic reports, prescriptions and administrative data, through networks that may be exposed to various security threats.

This report examines the application of symmetric-key cryptography, public-key cryptography, key-exchange mechanisms, digital signatures and hybrid encryption techniques in a multi-site healthcare network. It evaluates the characteristics and limitations of DES, 3DES, Blowfish and RC5, together with different block cipher modes of operation. Mathematical foundations relevant to RSA key generation, including the Extended Euclidean Algorithm, modular arithmetic, modular inverses and primality testing, are also examined.

The report further analyses RSA encryption, Diffie–Hellman key exchange and Elliptic Curve Cryptography (ECC), including their computational characteristics and security limitations. Digital signature mechanisms and authenticated hybrid encryption are evaluated to establish a secure approach for transmitting large healthcare files.

Based on the analysis and experimental demonstrations, a revised security architecture is proposed using authenticated TLS 1.3, ephemeral Elliptic-Curve Diffie–Hellman key exchange, HKDF-based key derivation, authenticated encryption and managed cryptographic keys. The proposed architecture also considers post-quantum cryptographic migration and operational security requirements.

The findings demonstrate that a hybrid cryptographic architecture provides an appropriate balance between confidentiality, integrity, authentication, computational efficiency and scalability for a multi-site healthcare network.

1. INTRODUCTION

Healthcare networks require secure communication between geographically distributed facilities. A central hospital may exchange medical information with several remote clinics through public or private communication networks. Such communication involves sensitive information that must remain confidential, accurate and accessible only to authorised users.

Traditional encryption approaches may not adequately address the requirements of a distributed healthcare environment when they are implemented without appropriate authentication, key management and integrity protection.

A secure healthcare communication architecture must satisfy the following requirements:

1. **Confidentiality:** Patient information must remain inaccessible to unauthorised individuals.

2. **Integrity:** Medical records and diagnostic information must not be modified during transmission without detection.
3. **Authentication:** Communicating hospitals, clinics and authorised devices must establish their identities.
4. **Scalability:** The cryptographic architecture must support large files and multiple simultaneously connected healthcare facilities.
5. **Key Management:** Cryptographic keys must be generated, distributed, stored, rotated and revoked securely.
6. **Availability:** Security mechanisms must minimise unnecessary processing overhead and support reliable healthcare operations.

This report evaluates relevant cryptographic algorithms and proposes an integrated security architecture that addresses these requirements.

Q1 – SYMMETRIC CRYPTOSYSTEM SELECTION AND MODES OF OPERATION

1.1 Evaluation of Symmetric Encryption Algorithms

Symmetric-key cryptography uses the same secret key for encryption and decryption. It is computationally efficient and is therefore suitable for encrypting large volumes of healthcare information.

The following algorithms are evaluated based on their internal structures, key characteristics, block sizes and security limitations.

Algorithm	Structure	Block Size	Key Characteristics	Security Considerations
DES	16-round Feistel network	64 bits	56 effective key bits	Vulnerable to brute-force attacks and unsuitable for modern security requirements.
3DES	Triple DES using Encrypt-Decrypt-Encrypt operations	64 bits	Two-key or three-key configurations	Provides greater key strength than DES but has significant performance limitations and a small block size.
Blowfish	16-round Feistel network	64 bits	Variable key length from 32 to 448 bits	Key setup is relatively expensive, and the 64-bit block size limits suitability for high-volume encryption.
RC5	Word-oriented encryption structure using modular addition, XOR and	Configurable	Configurable word size, number of rounds and key length	Flexible algorithm, but security depends on parameter selection and implementation quality.

	data-dependent rotations			

DES and Blowfish use Feistel network structures in which each round processes two data halves through a round function and subkey. 3DES applies the DES operation three times using the Encrypt-Decrypt-Encrypt sequence.

RC5 differs from traditional Feistel structures by employing modular arithmetic, exclusive-OR operations and data-dependent rotations.

Although these algorithms demonstrate different approaches to symmetric encryption, DES, 3DES and Blowfish are legacy algorithms and are not recommended for new healthcare deployments.

1.2 Comparison of Block Ciphers and Stream Ciphers

Symmetric encryption algorithms can be classified into block ciphers and stream ciphers.

Feature	Block Cipher	Stream Cipher
Operating principle	Processes data in fixed-size blocks.	Generates a keystream that is combined with plaintext.
Data processing	Requires an appropriate mode to process arbitrary-length messages.	Processes data continuously or in smaller segments.
Performance	Suitable for bulk data encryption.	Suitable for continuous data streams and low-latency communication.
Examples	AES, DES and Blowfish	ChaCha20
Security consideration	Requires correct mode selection and IV/nonce management.	Requires strict prevention of key and nonce reuse.

Block ciphers are commonly used for encrypting files and database records. Stream ciphers are useful for applications requiring continuous data processing.

For modern healthcare communication, AES-GCM and ChaCha20-Poly1305 are suitable authenticated-encryption choices.

1.3 Evaluation of Block Cipher Modes of Operation

Block cipher modes determine how a block cipher processes plaintext that may contain multiple blocks.

The following modes are examined.

Mode	Operating Principle	IV/Nonce Requirement	Security and Performance Characteristics

ECB	Encrypts each plaintext block independently.	Does not require an IV.	Reveals repeated plaintext patterns and is unsuitable for confidential structured data.
CBC	Combines each plaintext block with the preceding ciphertext block before encryption.	Requires a fresh, unpredictable IV.	Hides repeated block patterns when correctly implemented, but encryption is sequential.
CFB	Uses encrypted feedback from previous ciphertext to process subsequent plaintext.	Requires an appropriate fresh IV.	Supports stream-like processing, but encryption is sequential.
OFB	Generates a keystream by repeatedly encrypting an internal feedback value.	Requires a unique IV for each key.	Transmission errors do not propagate through subsequent plaintext bits, but keystream reuse is dangerous.
CTR	Generates a keystream by encrypting nonce and counter combinations.	Requires a unique nonce and non-repeating counter values for each key.	Supports parallel processing and random access. Nonce reuse compromises confidentiality.

ECB Mode

Electronic Codebook mode encrypts each block independently.

The same plaintext block encrypted using the same key produces the same ciphertext block. Consequently, repeated data patterns remain visible in the encrypted output.

This property makes ECB unsuitable for structured information such as medical records and database files.

CBC Mode

Cipher Block Chaining combines each plaintext block with the previous ciphertext block before encryption.

For the first block, an initialisation vector is used.

The encryption operation is represented as:

$C_1 = E_k(P_1 \oplus IV)$

$C_i = E_k(P_i \oplus C_{i-1})$

CBC requires an unpredictable IV and appropriate integrity protection. It does not provide authentication by itself.

CFB Mode

Cipher Feedback mode encrypts the previous feedback value and combines the result with plaintext to produce ciphertext.

This mode supports stream-oriented processing without requiring the plaintext to be processed exclusively as complete independent blocks.

OFB Mode

Output Feedback mode generates a sequence of encrypted feedback values that function as a keystream.

The plaintext is combined with this keystream using XOR.

Because the keystream is independent of ciphertext feedback, transmission errors do not propagate to subsequent plaintext segments.

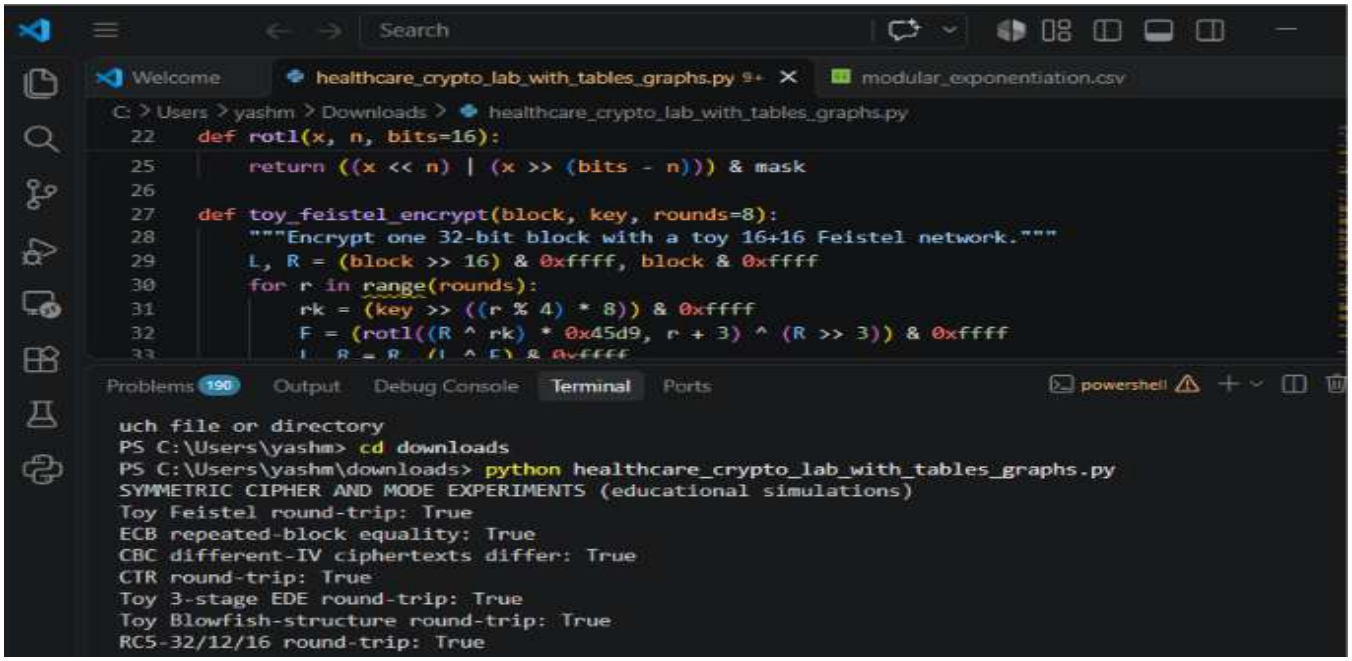
CTR Mode

Counter mode encrypts successive nonce-counter combinations to generate a keystream.

The generated keystream is combined with plaintext using XOR.

CTR supports parallel encryption and decryption. However, the same nonce and key combination must never be reused.

1.4 Implementation and Experimental Analysis



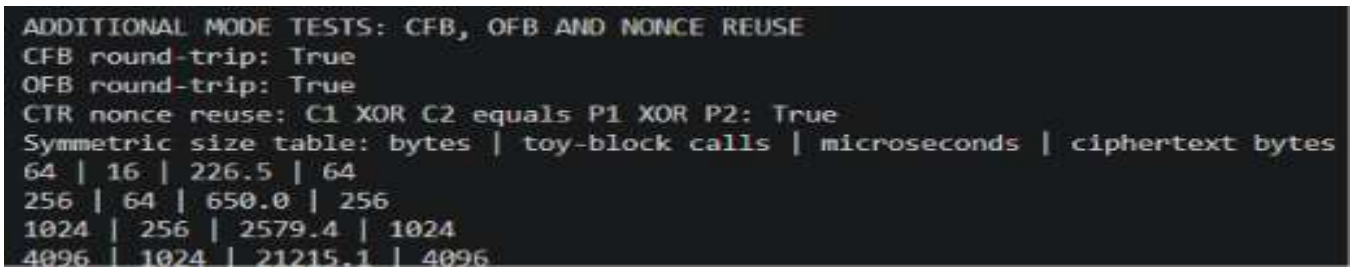


Figure 1. Q1 execution evidence: cipher round-trip checks, mode tests, nonce-reuse demonstration and message-size measurements.

The implementation evaluates encryption and decryption behaviour through educational cipher demonstrations.

The program includes RC5-32/12/16 and simplified Feistel-based demonstrations representing DES-like, 3DES-EDE-like and Blowfish-like structures.

The simplified structures are used to demonstrate cipher-round operations and encryption/decryption behaviour. They do not implement the complete official DES, 3DES or Blowfish specifications.

The following experiments are performed:

Experiment	Test Condition	Expected Observation
Feistel encryption	Encrypt a sample 32-bit data block.	Ciphertext is generated successfully.
Feistel decryption	Decrypt the generated ciphertext.	Original data is recovered.
3DES-EDE demonstration	Apply three encryption/decryption stages.	Original data is recovered.
Blowfish-structure demonstration	Apply the educational 16-round structure.	Original data is recovered.
RC5 encryption	Encrypt an 8-byte data block using a 16-byte key.	Ciphertext is generated.
RC5 decryption	Decrypt the encrypted block.	Original data is recovered.
ECB pattern analysis	Encrypt repeated plaintext blocks.	Repeated ciphertext blocks are observed.
CBC IV comparison	Encrypt identical plaintext using different IVs.	Different ciphertext outputs are generated.
CTR encryption	Encrypt and decrypt using a nonce-counter sequence.	Original plaintext is recovered.

The experiments demonstrate that encryption and decryption correctness depends on the selected algorithm and its corresponding mode of operation.

1.5 Nonce Reuse Analysis

Counter mode relies on a keystream generated from the encryption key and nonce-counter combination.

If the same key and nonce are reused, the same keystream is generated.

For two ciphertexts:

$C_1 = P_1 \oplus K$

$C_2 = P_2 \oplus K$

Combining both ciphertexts produces:

$C_1 \oplus C_2 = P_1 \oplus P_2$

This relationship exposes information about the corresponding plaintexts.

The implementation demonstrates this relationship using two different messages encrypted with the same key and nonce.

The experiment confirms that nonce uniqueness is essential for maintaining confidentiality when CTR mode is used.

1.6 Performance Analysis

The implementation measures processing time and the number of block operations for different message sizes.

Message Size (Bytes)	Toy Block Operations	Measured Processing Time (µs)	Ciphertext Size (Bytes)
64	16	66.69	64
256	64	414.01	256
1024	256	1312.24	1024
4096	1024	5976.48	4096

The number of block operations increases proportionally with message size.

The measured processing time also increases as the input size increases. These measurements represent the educational Python implementation and must not be interpreted as benchmarks of production cryptographic algorithms.

Graphical Analysis

The following graphs are generated from the experimental results:

1. Message Size versus Processing Time.
2. Message Size versus Number of Block Operations.

These graphs illustrate the relationship between input size and computational workload.

1.7 Algorithm Selection and Recommendation

Based on the evaluation, DES, 3DES and Blowfish are unsuitable for new healthcare deployments because of their security and performance limitations.

RC5 provides configurable parameters but is not the preferred standard for a modern healthcare communication architecture.

AES-256-GCM or ChaCha20-Poly1305 is recommended for bulk data encryption because these algorithms provide confidentiality and integrity through authenticated encryption.

The selection of a modern authenticated-encryption algorithm reduces the risk of unauthorised data modification and provides a suitable foundation for secure healthcare information exchange.

Q2 – NUMBER-THEORY ENGINE AND PRIME SELECTION FOR RSA

2.1 Mathematical Foundations

RSA cryptography relies on number-theoretic principles involving prime numbers, modular arithmetic, greatest common divisors and modular inverses.

Greatest Common Divisor

The greatest common divisor of two integers is the largest positive integer that divides both numbers without leaving a remainder.

It is represented as:

gcd(a,b)

The Euclidean Algorithm calculates the greatest common divisor by repeatedly applying the remainder operation.

Extended Euclidean Algorithm

The Extended Euclidean Algorithm extends the Euclidean Algorithm by determining integers x and y satisfying:

$ax + by = gcd(a,b)$

The algorithm is particularly important in RSA because it is used to calculate the modular inverse of the public exponent.

Modular Inverse

The modular inverse of a modulo m is an integer a^{-1} satisfying:

$a \times a^{-1} \equiv 1 \pmod{m}$

A modular inverse exists only when:

$\gcd(a,m) = 1$

This condition ensures that the public exponent has a valid multiplicative inverse modulo the RSA totient value.

Modular Exponentiation

Modular exponentiation calculates expressions of the form:

$a^e \bmod m$

The square-and-multiply algorithm performs this operation efficiently by repeatedly squaring the base and reducing intermediate results modulo m.

Its computational complexity is logarithmic in the exponent length.

2.2 RSA Key Generation Mathematics

RSA key generation involves the following steps:

1. Select two distinct large prime numbers p and q.
2. Calculate the modulus:

   $n = p \times q$

3. Calculate Euler's totient:

   $\varphi(n) = (p - 1)(q - 1)$

4. Select a public exponent e satisfying:

   $1 < e < \varphi(n)$

   $\gcd(e,\varphi(n)) = 1$

5. Calculate the private exponent d:

$$d = e^{-1} \bmod \varphi(n)$$

The resulting public key is:

(e,n)

The private key is:

(d,n)

The correctness of the RSA construction depends on selecting suitable primes and calculating the modular inverse correctly.

2.3 Primality Testing Techniques

Prime generation is an essential part of RSA key generation.

The following techniques are evaluated.

Method	Operating Principle	Advantages	Limitations
Trial Division	Tests possible divisors up to the square root of the candidate.	Deterministic for the tested integer range.	Computationally expensive for large integers.
Fermat Primality Test	Checks whether a^(n−1) mod n = 1 for selected bases.	Computationally simple.	Can incorrectly accept composite Carmichael numbers.
Miller–Rabin Test	Uses repeated modular exponentiation and witness testing.	Efficient probabilistic primality testing.	Requires appropriate bases and rounds to achieve a sufficiently low error probability.

Trial Division

Trial division checks whether a candidate integer is divisible by any integer between 2 and its square root.

If no divisor is found, the candidate is prime.

Fermat Primality Test

Fermat's Little Theorem states that if p is prime and gcd(a,p)=1, then:

a^(p−1) ≡ 1 (mod p)

The Fermat test uses this property to identify probable prime numbers.

However, some composite integers satisfy the Fermat congruence for many bases.

Miller–Rabin Primality Test

The Miller–Rabin test expresses n−1 in the form:

$n - 1 = 2^s \times d$

where d is odd.

It then performs modular exponentiation and repeated squaring to identify composite candidates.

Miller–Rabin is widely used as part of practical large-prime generation procedures.

2.4 Experimental Primality Results

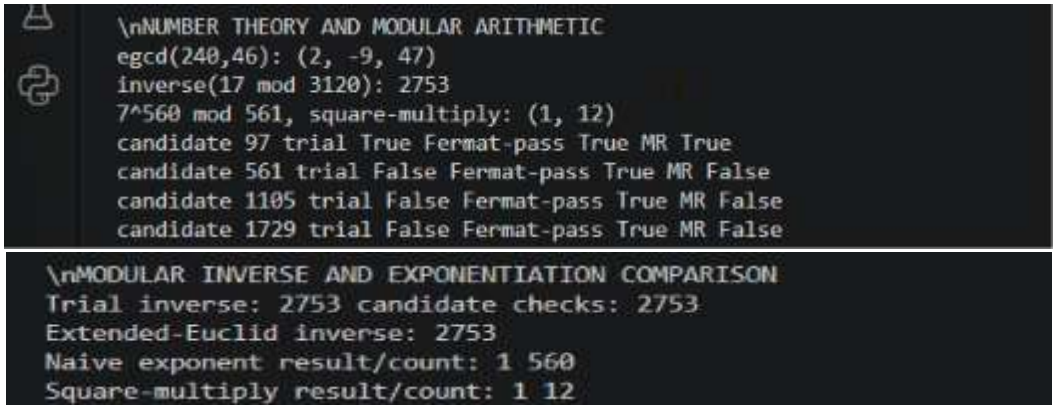


Figure 2. Q2 execution evidence: Extended Euclid, modular inverse, Carmichael-number comparison and exponentiation operation counts.

The implementation evaluates the following integers.

Candidate	Trial Division	Fermat Test (Base 2)	Miller–Rabin
97	Prime	Pass	Pass
561	Composite	Fail	Fail
1105	Composite	Fail	Fail
1729	Composite	Pass	Fail

The values 561, 1105 and 1729 are Carmichael numbers.

The experiment demonstrates that the Fermat test may accept certain composite numbers, whereas Miller–Rabin can identify these candidates as composite using the selected test bases.

This observation establishes the importance of using stronger primality-testing techniques during RSA key generation.

2.5 Modular Inverse and Exponentiation Comparison

For the RSA numerical example:

a = 17

m = 3120

The Extended Euclidean Algorithm calculates:

$17 \times 2753 \equiv 1 \pmod{3120}$

Therefore:

$17^{-1} \bmod 3120 = 2753$

The implementation also compares trial-search modular inverse calculation with the Extended Euclidean Algorithm.

Method	Result	Computational Observation
Trial-search inverse	2753	Requires repeated candidate testing.
Extended Euclidean Algorithm	2753	Uses logarithmically growing division steps.
Naive modular exponentiation	1	Uses 560 modular multiplications for the specified demonstration.
Square-and-multiply	1	Uses 12 modular multiplications for the specified demonstration.

The results demonstrate that square-and-multiply significantly reduces the number of modular multiplication operations compared with naive repeated multiplication.

2.6 Prime Density and RSA Key Selection

The Prime Number Theorem provides an approximation for the probability that a sufficiently large integer near x is prime.

The approximate probability is:

$P(\text{prime}) \approx 1 / \ln(x)$

This relationship indicates that larger candidate integers require more attempts on average before a prime is identified.

For practical RSA key generation, secure random-number generation and efficient primality testing are essential.

A suitable key-generation procedure includes:

- Cryptographically secure random prime candidate generation.
- Rejection of candidates divisible by small prime numbers.
- Miller–Rabin testing with appropriate independent bases.
- Verification of the relationship between the public exponent and Euler's totient.
- Protection of generated private keys.

RSA-2048 is commonly used as a minimum baseline for new classical RSA deployments, while RSA-3072 provides a higher classical security margin.

The use of vetted cryptographic libraries is recommended for production key generation.

Q3 – RSA ENCRYPTION, DECRYPTION AND SECURITY ANALYSIS

3.1 RSA Key Generation

RSA is an asymmetric cryptographic algorithm that uses a public key for encryption and a private key for decryption.

The algorithm relies on the computational difficulty of factoring sufficiently large integers into their prime factors.

The following numerical example demonstrates the RSA key-generation process.

Select:

$p = 61$

$q = 53$

Calculate the modulus:

$n = p \times q$

$n = 61 \times 53$

$n = 3233$

Calculate Euler's totient:

$\varphi(n) = (p - 1)(q - 1)$

$\varphi(n) = 60 \times 52$

$\varphi(n) = 3120$

Select the public exponent:

$e = 17$

Since:

$\gcd(17,3120) = 1$

the selected public exponent is valid.

Calculate the private exponent:

$d = 2753$

This satisfies:

$17 \times 2753 \equiv 1 \pmod{3120}$

Therefore, the resulting keys are:

Public Key: (17,3233)

Private Key: (2753,3233)

3.2 RSA Encryption and Decryption

RSA encryption is performed using:

$C = M^e \bmod n$

RSA decryption is performed using:

$M = C^d \bmod n$

where:

- M represents the plaintext integer.
- C represents the ciphertext integer.
- e represents the public exponent.

- d represents the private exponent.
- n represents the RSA modulus.

For the selected example:

M = 65

Encryption:

$C = 65^{17} \bmod 3233$

C = 2790

Decryption:

$M = 2790^{2753} \bmod 3233$

M = 65

The original plaintext value is successfully recovered.

3.3 RSA Experimental Results

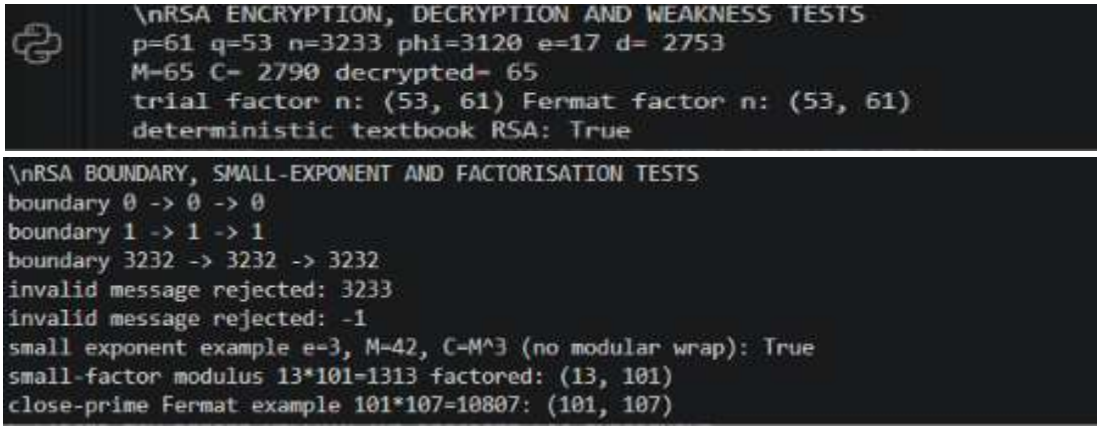


Figure 3. Q3 execution evidence: hand-checkable RSA example, deterministic behaviour, boundary validation and toy factorisation.

Parameter	Value
Prime p	61
Prime q	53
Modulus n	3233
Euler's Totient φ(n)	3120
Public Exponent e	17
Private Exponent d	2753
Original Message M	65
Encrypted Message C	2790

Decrypted Message	65

The experimental results confirm that the RSA encryption and decryption operations recover the original message when the correct key pair is used.

3.4 RSA Security Weaknesses

Although RSA provides asymmetric encryption, several implementation and configuration weaknesses can compromise its security.

Small Key Sizes

Small RSA moduli can be factored using computational methods.

Once the modulus is factored, an attacker can calculate Euler's totient and derive the private exponent.

Weak Prime Generation

Predictable, repeated or insufficiently random prime numbers can weaken RSA key generation.

Textbook RSA

Textbook RSA directly encrypts message representatives without randomised padding.

Consequently, identical messages encrypted using the same key produce identical ciphertexts.

This deterministic behaviour may expose information about the plaintext.

Small-Exponent Attacks

An improperly padded RSA implementation using a small public exponent may expose small messages to mathematical attacks.

Side-Channel Attacks

Timing variations, power consumption and other implementation characteristics may reveal information about private-key operations.

3.5 Factorisation Demonstration

The implementation demonstrates factorisation using small numerical examples.

For the modulus:

n = 3233

the factors are:

53 × 61 = 3233

The private exponent can then be calculated because the prime factors are known.

The experiment also demonstrates factorisation involving small factors and closely spaced primes.

These experiments illustrate weaknesses associated with small RSA moduli. They do not represent the computational difficulty of factoring properly generated RSA-2048 or RSA-3072 moduli.

3.6 RSA-OAEP and Secure Deployment

RSA-OAEP provides a randomised encoding mechanism for RSA encryption.

It prevents identical plaintext messages from producing identical ciphertexts when correctly implemented.

RSA-OAEP with SHA-256 is suitable when RSA encryption is required for protecting small cryptographic values, such as encryption keys.

However, RSA should not be used to encrypt large healthcare files directly.

For large data transmission, symmetric encryption should be used together with an authenticated key-establishment or key-wrapping mechanism.

Q4 – DIFFIE–HELLMAN KEY EXCHANGE AND KEY MANAGEMENT

4.1 Diffie–Hellman Key Exchange

Diffie–Hellman is a public-key key-agreement mechanism that enables two communicating parties to establish a common secret over an observable communication channel.

The security of the traditional finite-field Diffie–Hellman construction is based on the computational difficulty of the discrete logarithm problem.

The two communicating parties agree on public parameters:

- p: a prime number.
- g: a generator of the selected group.

Clinic A selects a private value a and calculates:

$A = g^a \bmod p$

Clinic B selects a private value b and calculates:

$B = g^b \bmod p$

Both parties exchange their public values.

Clinic A calculates:

$K_a = B^a \bmod p$

Clinic B calculates:

$K_\beta = A^b \bmod p$

Both calculations produce:

$K = g^{ab} \bmod p$

Therefore:

$K_a = K_\beta$

The shared secret can subsequently be processed through a Key Derivation Function to produce cryptographic session keys.

4.2 Diffie–Hellman Numerical Demonstration

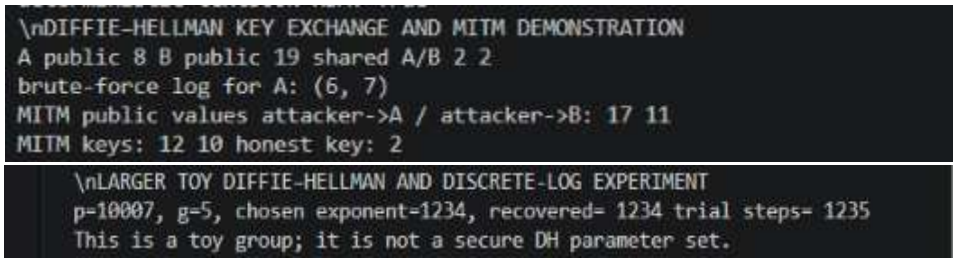


Figure 4. Q4 execution evidence: shared-secret agreement, unauthenticated MITM demonstration and toy discrete-log search.

The following parameters are selected:

$p = 23$

$g = 5$

Private value of Clinic A:

a = 6

Private value of Clinic B:

b = 15

Clinic A calculates:

$A = 5^6 \bmod 23$

A = 8

Clinic B calculates:

$B = 5^{15} \bmod 23$

B = 19

Clinic A calculates the shared secret:

$K_a = 19^6 \bmod 23$

$K_a = 2$

Clinic B calculates the shared secret:

$K_\beta = 8^{15} \bmod 23$

$K_\beta = 2$

Therefore:

$K_a = K_\beta = 2$

The experiment confirms that both communicating parties derive the same shared secret.

4.3 Man-in-the-Middle Attack

Traditional unauthenticated Diffie–Hellman key exchange does not independently verify the identities of communicating parties.

An attacker positioned between two communicating healthcare facilities may intercept and replace exchanged public values.

The attacker establishes one shared secret with Clinic A and another shared secret with Clinic B.

Consequently, the attacker may decrypt information received from one party and re-encrypt it before forwarding it to the other party.

This attack is known as a Man-in-the-Middle (MITM) attack.

Attack Mitigation

The following mechanisms are recommended:

1. Certificate-based authentication.
2. Digital signatures over the key-exchange transcript.
3. Certificate-chain validation.
4. Hostname and identity verification.
5. Certificate revocation checking.
6. Authenticated TLS 1.3 communication.

These mechanisms establish the identity of communicating parties and prevent unauthorised substitution of key-exchange values.

4.4 Larger Parameter Demonstration

The implementation also performs a discrete-logarithm search using a larger educational parameter set.

The demonstration uses:

p = 10007

g = 5

The selected private exponent is:

a = 1234

The implementation recovers the exponent after 1,235 candidate checks.

This experiment illustrates the relationship between the search space and the computational effort required for a brute-force discrete-logarithm search.

The selected parameters are intended only for academic demonstration and are not suitable for real cryptographic communication.

4.5 Cryptographic Key Management

Secure key management is essential for maintaining the confidentiality and authenticity of healthcare communications.

Key Management Stage	Recommended Security Measure
Key Generation	Use cryptographically secure random-number generators.
Key Storage	Store long-term private keys in Hardware Security Modules or managed Key Management Systems.
Key Distribution	Use authenticated TLS communication and certificate-based identity verification.
Key Rotation	Rotate keys according to defined organisational policies and immediately after suspected compromise.
Key Revocation	Revoke certificates associated with compromised or decommissioned devices.
Key Recovery	Maintain protected backups where recovery is required, with strict access controls.
Session Management	Use ephemeral key exchange and appropriate session-key lifetimes.
Audit and Monitoring	Maintain records of key-management operations and privileged access.

A centralised key-management architecture provides consistent control over cryptographic identities across distributed healthcare facilities.

Q5 – ELLIPTIC CURVE CRYPTOGRAPHY AND PUBLIC-KEY STRATEGY COMPARISON

5.1 Mathematical Foundation of ECC

Elliptic Curve Cryptography is a public-key cryptographic technique based on mathematical operations over elliptic curves defined on finite fields.

A commonly used elliptic curve equation is:

$y^2 \equiv x^3 + ax + b \pmod{p}$

where:

- a and b are curve parameters.
- p is a prime modulus.
- x and y represent point coordinates.

The curve must satisfy the non-singularity condition:

$4a^3 + 27b^2 \neq 0 \pmod{p}$

Points satisfying the curve equation form a mathematical group under elliptic-curve point addition.

Scalar multiplication involves repeatedly adding a point to itself.

If G is a base point and d is a private scalar, the corresponding public key is:

$Q = dG$

The security of ECC relies on the difficulty of determining d when G and Q are known.

5.2 ECC Key Agreement

In Elliptic Curve Diffie–Hellman (ECDH), two communicating parties establish a shared secret using their private scalars and exchanged public points.

Clinic A selects private scalar d_a and calculates:

$Q_a = d_aG$

Clinic B selects private scalar d_β and calculates:

$Q_\beta = d_\beta G$

The parties exchange their public points.

Clinic A calculates:

$S_a = d_aQ_\beta$

Clinic B calculates:

$S_\beta = d_\beta Q_a$

Both obtain:

$S = d_ad_\beta G$

Therefore:

$S_a = S_\beta$

5.3 ECC Numerical Demonstration



Figure 5. Q5 execution evidence: both participants derive the same toy ECC shared point.

The implementation uses the following educational elliptic curve:

$y^2 = x^3 + 2x + 2 \pmod{17}$

The selected base point is:

$G = (5,1)$

Private scalar of Clinic A:

$d_a = 5$

Private scalar of Clinic B:

$d_\beta = 7$

The public points are:

$Q_a = (9,16)$

$Q_\beta = (0,6)$

Both parties calculate the same shared point:

$S = (10,11)$

The result confirms the correctness of the implemented point-addition and scalar-multiplication operations for the selected example.

The curve parameters are intentionally small and are not suitable for cryptographic deployment.

5.4 Comparison of RSA, Diffie–Hellman and ECC

Feature	RSA	Diffie–Hellman	ECC
Mathematical Basis	Integer factorisation	Discrete logarithm	Elliptic-curve discrete logarithm
Primary Purpose	Encryption and digital signatures	Key agreement	Key agreement and digital signatures

Key Size	Relatively large	Relatively large	Smaller for comparable classical security levels
Computational Characteristics	Public and private operations have different computational costs	Requires modular exponentiation	Efficient scalar multiplication with compact parameters
Bandwidth	Larger public-key material	Large public values	Smaller public-key material
Healthcare Suitability	Suitable for selected legacy and compatibility requirements	Suitable when authenticated and correctly configured	Suitable for modern authenticated key establishment

ECC provides comparable classical security with smaller key sizes than traditional RSA and finite-field Diffie–Hellman systems.

This characteristic makes ECC suitable for healthcare environments containing devices with limited computational and memory resources.

5.5 Recommended Public-Key Strategy

The proposed architecture uses authenticated ephemeral ECDHE for session-key establishment.

X25519 or an approved NIST elliptic curve may be selected according to the organisation's security policy and platform support.

Long-term private keys should be protected using appropriate hardware-backed or managed key-storage mechanisms.

Public-key operations should be limited to authentication and key establishment wherever possible, while symmetric authenticated encryption should be used for bulk data protection.

Q6 – DIGITAL SIGNATURES, HYBRID ENCRYPTION AND QUANTUM READINESS

6.1 Digital Signature Mechanisms

Digital signatures provide mechanisms for verifying message integrity and authenticating the origin of digital information.

A digital signature is generated using the sender's private signing key and verified using the corresponding public key.

The signing process involves the following operations:

1. Calculate a cryptographic hash of the original message.
2. Generate a digital signature using the sender's private key and the selected signature algorithm.

3. Transmit the message, signature and required certificate information.
4. Verify the certificate and sender identity.
5. Verify the signature using the sender's public key.
6. Accept the message only when the verification process succeeds.

The following signature mechanisms are evaluated.

Algorithm	Mathematical Basis	Characteristics
RSA-PSS	Integer factorisation	Provides RSA-based digital signatures with probabilistic encoding.
DSA/DSS	Finite-field discrete logarithm	Requires secure generation and handling of per-message nonce values.
ECDSA	Elliptic-curve discrete logarithm	Provides compact signatures and efficient verification in supported implementations.

RSA-PSS is recommended for RSA-based digital signatures.

ECDSA is suitable where elliptic-curve-based signatures are required.

For DSA and ECDSA, incorrect nonce generation, reuse or bias can expose private signing keys.

6.2 Authenticated Hybrid Encryption

Hybrid encryption combines public-key cryptography and symmetric-key cryptography to provide secure and efficient transmission of large healthcare files.

The proposed approach is suitable for transmitting patient records, prescriptions, diagnostic reports and other confidential medical documents between healthcare facilities.

The encryption process consists of the following steps:

1. **Data Encryption Key Generation:** Generate a fresh cryptographically secure random symmetric Data Encryption Key (DEK).
2. **File Encryption:** Encrypt the healthcare file using AES-256-GCM or ChaCha20-Poly1305 with a unique nonce. Include authenticated metadata such as clinic identification, patient-record reference, timestamp and file-format version.
3. **Key Protection:** Protect the DEK for the intended recipient using an approved key-wrapping mechanism or Key Management System (KMS). Alternatively, establish session keys through authenticated ephemeral ECDHE and a Key Derivation Function (KDF).
4. **Digital Signature:** Generate a digital signature over the canonical file manifest or file digest when independent sender authentication and integrity verification are required.
5. **Secure Transmission:** Transmit the ciphertext, nonce, authentication tag, protected DEK or key reference, manifest and digital signature to the intended recipient.

This approach provides confidentiality, integrity and authenticity while reducing the computational overhead associated with applying public-key cryptography directly to large files.

6.3 Hybrid Encryption and Decryption Process

The receiving healthcare facility performs the following operations:

1. Validate the sender's certificate and digital signature.
2. Obtain or securely unwrap the Data Encryption Key.
3. Authenticate the ciphertext and associated metadata.
4. Verify the authentication tag before releasing the decrypted information.
5. Verify the file digest and manifest.
6. Record the transaction in the security audit system.

Authenticated associated data should bind the encrypted content to relevant identifiers, such as the clinic ID, record ID and file version.

This prevents encrypted data from being substituted into an unrelated communication context.

6.4 Experimental Tampering Analysis



Figure 6. Q6 execution evidence: signature verification, modified-message rejection, hybrid tamper detection and wrong-key rejection.

The implementation evaluates message modification, signature verification and hybrid encryption integrity using educational cryptographic demonstrations.

Experiment	Expected Result	Observed Result
Original message verification	Signature verification should succeed.	Verification succeeds.
Modified message	Signature verification should fail.	Verification fails.
Modified ciphertext	Authentication verification should reject the ciphertext.	Tampered ciphertext is rejected.
Incorrect session key	Authentication verification should fail.	Verification fails.
Original hybrid message	Decryption should recover the original message.	Original message is recovered.

The supplied educational hybrid demonstration uses a SHA-256-derived XOR keystream and HMAC for integrity verification.

This construction is included only to demonstrate encryption, authentication and tampering detection. It is not a standard authenticated-encryption algorithm and must not be used for production healthcare information.

A production deployment must use a vetted AEAD implementation, such as AES-GCM or ChaCha20-Poly1305.

6.5 Direct RSA Encryption Versus Hybrid Encryption

Characteristic	Direct RSA Encryption	Hybrid Encryption
Encryption approach	Applies public-key operations to individual message representatives.	Uses symmetric encryption for the payload and public-key mechanisms for key establishment or key protection.
Computational efficiency	Inefficient for large data.	Efficient for large files.
Message size	Limited by the RSA modulus and encoding requirements.	Suitable for large data volumes.
Scalability	Poor choice for bulk file encryption.	Suitable for distributed healthcare systems.
Recommended application	Protecting small values or signing data, depending on the RSA scheme.	Protecting healthcare files and communication sessions.

The educational operation-count comparison uses a small RSA modulus and one-byte message representatives.

Approach	Estimated Public-Key Operations for a 1 KiB Demonstration File
Direct educational RSA approach	Approximately 1,024 RSA encryption operations
Hybrid approach	One key-protection operation, followed by symmetric payload encryption

The comparison illustrates why public-key cryptography should not be applied independently to every block of a large healthcare file.

In a production architecture, the symmetric algorithm processes the payload, while public-key mechanisms are used for authenticated key establishment, key wrapping or digital signatures.

6.6 Quantum Cryptography and Post-Quantum Security

Quantum computing introduces potential risks to existing public-key cryptographic systems.

Shor's algorithm can theoretically solve integer factorisation and discrete logarithm problems efficiently on a sufficiently capable fault-tolerant quantum computer.

Consequently, RSA and elliptic-curve cryptographic systems would be vulnerable to such attacks.

Grover's algorithm provides a quadratic speed-up for unstructured search, affecting the security margin of symmetric cryptographic algorithms.

Quantum Key Distribution

Quantum Key Distribution (QKD) uses quantum communication principles to establish shared cryptographic key material.

The measurement of quantum states may introduce detectable disturbances when an eavesdropper attempts to intercept the communication.

However, QKD requires specialised infrastructure and authenticated classical communication.

It does not replace endpoint authentication, certificate management, secure software or cryptographic key storage.

Post-Quantum Migration Strategy

The healthcare organisation should adopt the following migration approach:

1. Identify all systems that depend on RSA and ECC.
2. Classify sensitive healthcare information according to its confidentiality lifetime.
3. Prioritise systems containing long-term confidential patient information.
4. Evaluate standards-based post-quantum key-establishment and signature mechanisms.
5. Introduce cryptographic agility to support algorithm replacement.
6. Test hybrid classical and post-quantum key-establishment mechanisms where supported.
7. Establish a migration schedule for certificates, devices and cryptographic infrastructure.

NIST standards FIPS 203 and FIPS 204 provide standardised post-quantum mechanisms for key encapsulation and digital signatures.

Q7 – INTEGRATED SECURITY DESIGN: DIAGNOSIS, REDESIGN AND VALIDATION

7.1 Identification of Existing Security Weaknesses

The initial healthcare communication design presents several security and performance limitations.

The use of unauthenticated Diffie–Hellman key exchange allows an attacker to perform a Man-in-the-Middle attack by substituting public values.

Textbook RSA encryption produces deterministic ciphertexts for repeated plaintext inputs and does not provide the protections expected from modern encryption schemes.

ECB mode also reveals repeated plaintext patterns because identical plaintext blocks generate identical ciphertext blocks.

Furthermore, applying public-key operations directly to large files introduces unnecessary computational overhead.

The experimental demonstrations confirm these weaknesses through repeated-message encryption, nonce-reuse analysis, ECB pattern comparison and the Diffie–Hellman MITM demonstration.

7.2 Proposed Redesign Objectives

The revised architecture aims to achieve the following objectives:

1. Establish authenticated communication between all healthcare facilities.
2. Prevent unauthorised substitution of key-exchange values.
3. Protect large healthcare files using authenticated symmetric encryption.
4. Ensure that modifications to encrypted information are detected.
5. Reduce unnecessary public-key processing.
6. Maintain secure cryptographic key management.
7. Support future migration to post-quantum cryptographic mechanisms.

7.3 Revised Healthcare Security Architecture

The proposed architecture contains the following components:

Clinic Device Identity

Each healthcare facility is assigned a unique digital identity through a certificate-based Public Key Infrastructure.

Private identity keys should be protected using TPM-backed or HSM-backed storage where supported.

Authenticated Communication

TLS 1.3 is used to establish secure communication between healthcare facilities and the central hospital.

Authenticated ephemeral ECDHE provides session-key establishment and forward secrecy when implemented with the required authentication and key-management protections.

Session-Key Derivation

HKDF is used as part of the TLS key schedule to derive cryptographic traffic keys from the established secret and handshake context.

Bulk Data Encryption

AES-256-GCM or ChaCha20-Poly1305 is used to encrypt healthcare records and diagnostic files.

Unique nonces and authenticated associated data are required.

Digital Signature Verification

Digital signatures are used for selected records and manifests where independent sender authentication and integrity verification are required.

Centralised Key Management

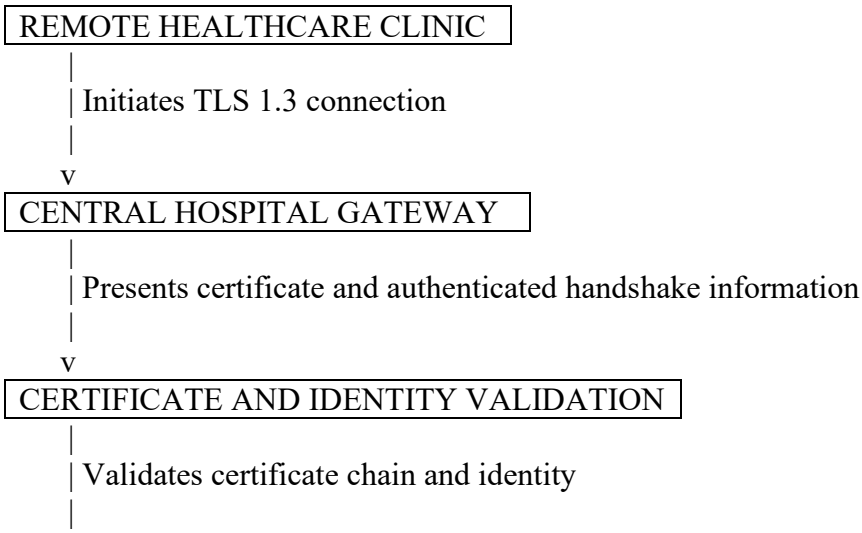
A centralised KMS or HSM-backed architecture manages long-term keys, certificates, rotation and revocation.

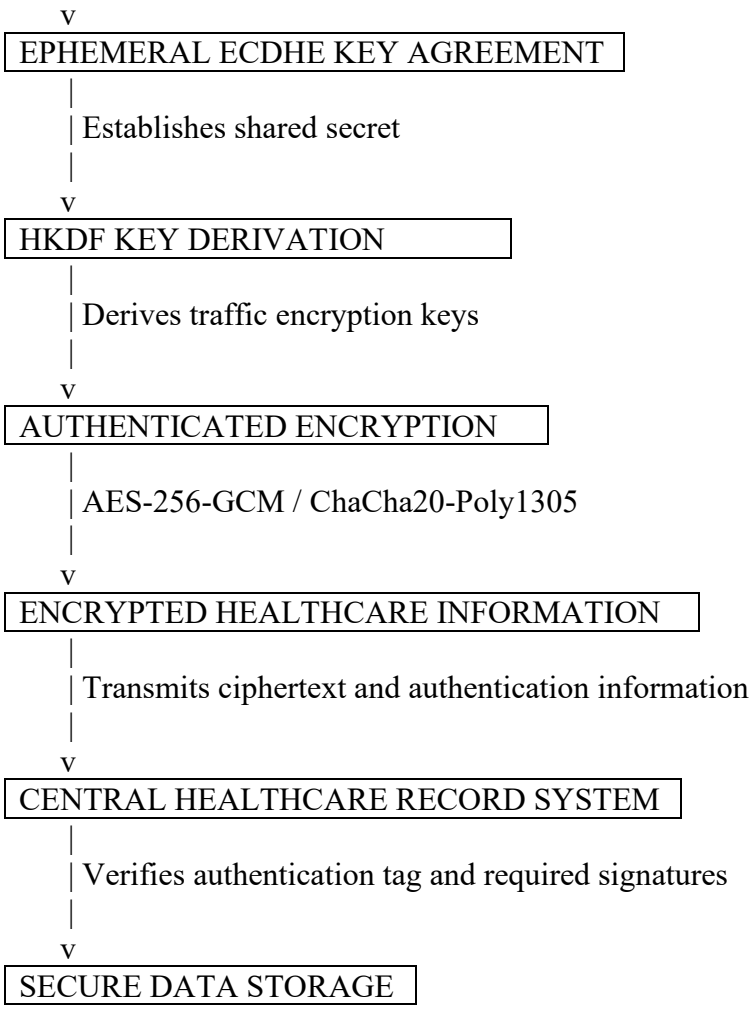
Audit and Monitoring

The system maintains audit records for authentication, key-management operations and access to sensitive healthcare information.

7.4 Proposed Communication Sequence

The following sequence illustrates the proposed communication process.





7.5 Mathematical and Security Justification

The revised architecture uses established cryptographic principles to address the weaknesses identified in the initial design.

Authenticated ECDHE establishes a shared secret between verified communicating parties.

HKDF derives context-specific traffic keys from the established secret.

Authenticated encryption provides confidentiality and integrity for the transmitted information.

Digital signatures provide independent verification of the sender and the integrity of signed manifests.

The use of a symmetric authenticated-encryption algorithm for bulk data reduces computational overhead compared with attempting to apply public-key encryption to individual payload blocks.

7.6 Experimental Validation

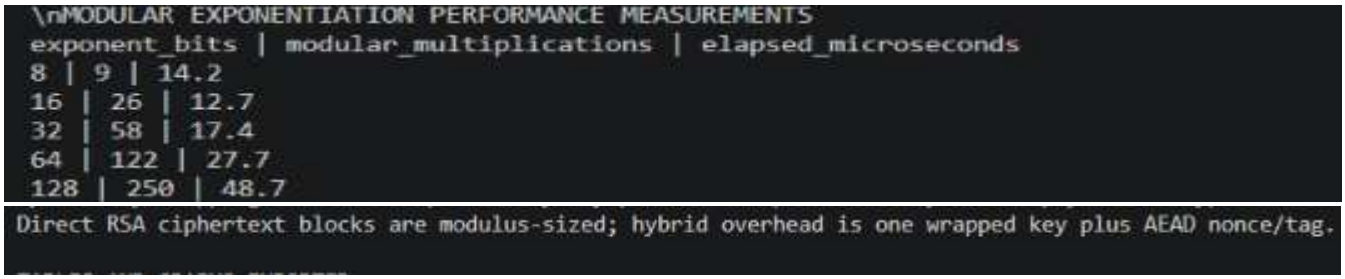


Figure 7. Q7 execution evidence: modular-exponentiation operation counts and the direct-RSA versus hybrid comparison.

The implementation evaluates the relevant mathematical and security properties using educational demonstrations.

Validation Test	Expected Behaviour	Experimental Observation
RSA encryption and decryption	Original plaintext should be recovered.	Message 65 was recovered successfully.
Repeated textbook RSA encryption	Identical inputs produce identical ciphertexts.	Deterministic behaviour confirmed.
Diffie–Hellman exchange	Both participants calculate the same shared secret.	Both participants calculated 2.
MITM demonstration	Attacker establishes separate shared secrets with the endpoints.	Separate attacker-controlled keys were generated.
ECC key agreement	Both participants calculate the same shared point.	Both participants calculated (10,11).
Ciphertext tampering	Modified ciphertext must fail authentication.	Tampering was detected.
Modular exponentiation comparison	Square-and-multiply should require fewer multiplications than naive exponentiation.	The implementation demonstrated reduced operation counts.

The experiments provide evidence of the mathematical properties and security limitations discussed in the report.

They do not constitute a complete production TLS implementation or a benchmark of production-grade cryptographic libraries.

7.7 Performance and Scalability Considerations

The revised architecture separates key establishment from bulk data encryption.

Public-key operations are used for authentication, key agreement and selected key-protection operations.

Symmetric authenticated encryption processes the healthcare information itself.

This division reduces unnecessary public-key processing and supports efficient transmission of large medical files.

The architecture also supports multiple clinics through certificate-based device identities and centralised key management.

7.8 Residual Risks and Mitigation Measures

Although the proposed architecture improves security, several operational risks remain.

Residual Risk	Potential Impact	Mitigation
Endpoint compromise	Exposure of decrypted patient information.	Secure boot, endpoint protection and regular security updates.
Private-key compromise	Unauthorised impersonation.	HSM/TPM protection, key rotation and certificate revocation.
Certificate authority compromise	Unauthorised certificates may be issued.	CA protection, restricted privileges and certificate monitoring.
Weak randomness	Predictable cryptographic keys.	Approved cryptographically secure random-number generators.
Nonce reuse	Compromise of authenticated-encryption security.	Strict nonce management and per-key uniqueness controls.
Denial-of-Service attacks	Interruption of healthcare communication.	Network monitoring, rate limiting and availability controls.
Implementation vulnerabilities	Exposure through software defects.	Security testing, code review and vetted cryptographic libraries.
Quantum computing threats	Future compromise of RSA and ECC.	Post-quantum migration planning and cryptographic agility.

7.9 Final Security Recommendation

The proposed hybrid cryptographic architecture is recommended for the multi-site healthcare network.

It combines authenticated key establishment, modern symmetric encryption, certificate-based authentication, digital signatures and centralised cryptographic key management.

The architecture addresses the identified weaknesses of unauthenticated Diffie–Hellman, textbook RSA and ECB mode.

It also provides a structured migration approach for future post-quantum security requirements.

The design provides an appropriate balance between confidentiality, integrity, authentication, performance and scalability for healthcare information exchange.

8. CONCLUSION

This report evaluated symmetric and asymmetric cryptographic techniques for securing communication within a multi-site healthcare network.

The analysis demonstrated the limitations of legacy symmetric encryption algorithms and the importance of selecting appropriate block cipher modes. The mathematical foundations of RSA key generation, including modular arithmetic, modular inverses and primality testing, were examined through numerical demonstrations.

RSA encryption and decryption, Diffie–Hellman key exchange and Elliptic Curve Cryptography were evaluated to understand their mathematical properties, computational characteristics and security limitations.

The analysis also established the importance of digital signatures, authenticated encryption and secure key management in protecting healthcare information.

The proposed architecture combines TLS 1.3, authenticated ephemeral ECDHE, HKDF, AES-256-GCM or ChaCha20-Poly1305, certificate-based authentication and centralised key management.

The experimental demonstrations support the identified security weaknesses and the recommended design principles. The report also considers post-quantum migration requirements to support the long-term protection of sensitive healthcare information.

Overall, an authenticated hybrid cryptographic architecture provides an effective approach to securing communication between distributed healthcare facilities while maintaining confidentiality, integrity, authentication and operational efficiency.

9. REFERENCES

1. National Institute of Standards and Technology (NIST), *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*, Special Publication 800-56A Revision 3.
2. National Institute of Standards and Technology (NIST), *Recommendation for Key Management: Part 1 – General*, Special Publication 800-57 Part 1 Revision 5.
3. National Institute of Standards and Technology (NIST), *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, Special Publication 800-38D.
4. Rescorla, E., *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, Internet Engineering Task Force, 2018.
5. Moriarty, K., Kaliski, B., Jonsson, J. and Rusch, A., *PKCS #1: RSA Cryptography Specifications Version 2.2*, RFC 8017, Internet Engineering Task Force, 2016.
6. National Institute of Standards and Technology (NIST), *Digital Signature Standard (DSS)*, FIPS 186-5, 2023.

7. National Institute of Standards and Technology (NIST), *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, 2024.
8. National Institute of Standards and Technology (NIST), *Module-Lattice-Based Digital Signature Standard*, FIPS 204, 2024.

APPENDIX A – IMPLEMENTATION AND EXPERIMENTAL EVIDENCE

Evidence note: The terminal-style panels in this appendix are illustrative representations of program output; they are not direct screenshots of the student's local VS Code or terminal session. The Python implementation should be executed on the student's computer, and genuine screenshots and environment details should be recorded before submission.

A.1 Implementation Environment

The experimental implementation is performed using Python.

The following information should be recorded from the actual execution environment:

Parameter	Recorded Value
Operating System	To be recorded
Python Version	To be recorded
Processor	To be recorded
Execution Date	To be recorded
Implementation File	healthcare_crypto_lab_with_tables_graphs.py

A.2 Execution Procedure

The implementation is executed using:

python healthcare_crypto_lab_with_tables_graphs.py

The Python version can be verified using:

python --version

The program output should be captured and included as experimental evidence.

A.3 Additional Experimental Results

RSA Boundary Validation

The implementation evaluates the following message representatives:

Input	Expected Behaviour
M = 0	Valid representative

M = 1	Valid representative
M = n − 1	Valid representative
M = n	Rejected
M = −1	Rejected

Diffie–Hellman Brute-Force Demonstration

The implementation evaluates a larger educational group using:

p = 10007

g = 5

Selected private exponent:

a = 1234

The implementation recovers the selected exponent after 1,235 candidate checks.

This result demonstrates the relationship between the search range and the computational effort required by a brute-force discrete-logarithm search.

Modular Exponentiation Operation Counts

Exponent Length	Modular Multiplications
8 bits	9
16 bits	26
32 bits	58
64 bits	122
128 bits	250

The measurements depend on the selected exponent values and the implementation's operation-counting method.

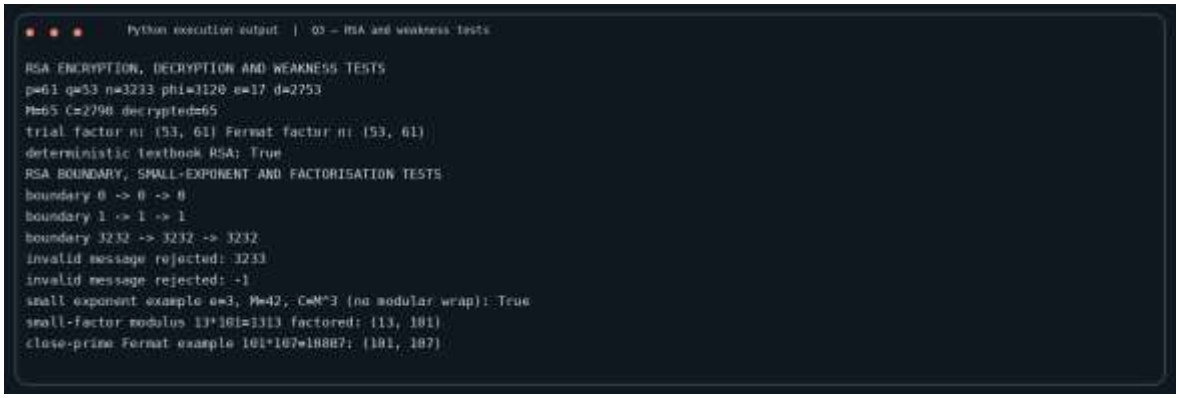
A.4 Execution Output and Experimental Evidence



Appendix Figure A1. Symmetric cipher and mode execution output.



Appendix Figure A2. Number-theory and primality-test execution output.



Appendix Figure A3. RSA encryption, boundary and factorisation execution output.



Appendix Figure A4. Diffie–Hellman and MITM execution output.



Appendix Figure A5. ECC key-agreement execution output.

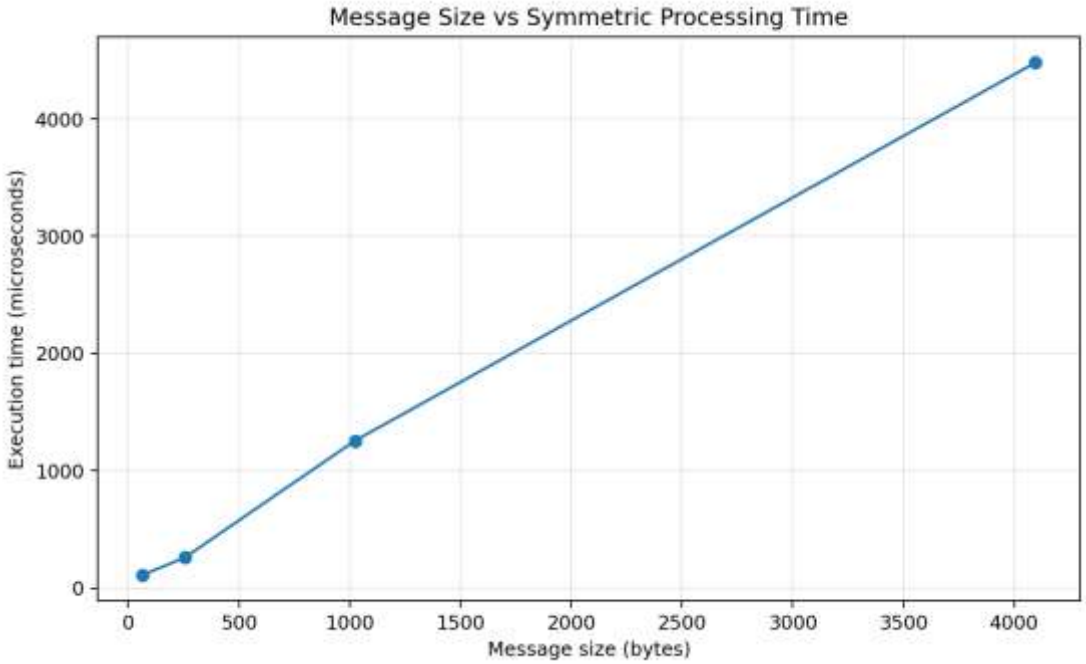


Appendix Figure A6. Digital-signature and hybrid-encryption execution output.

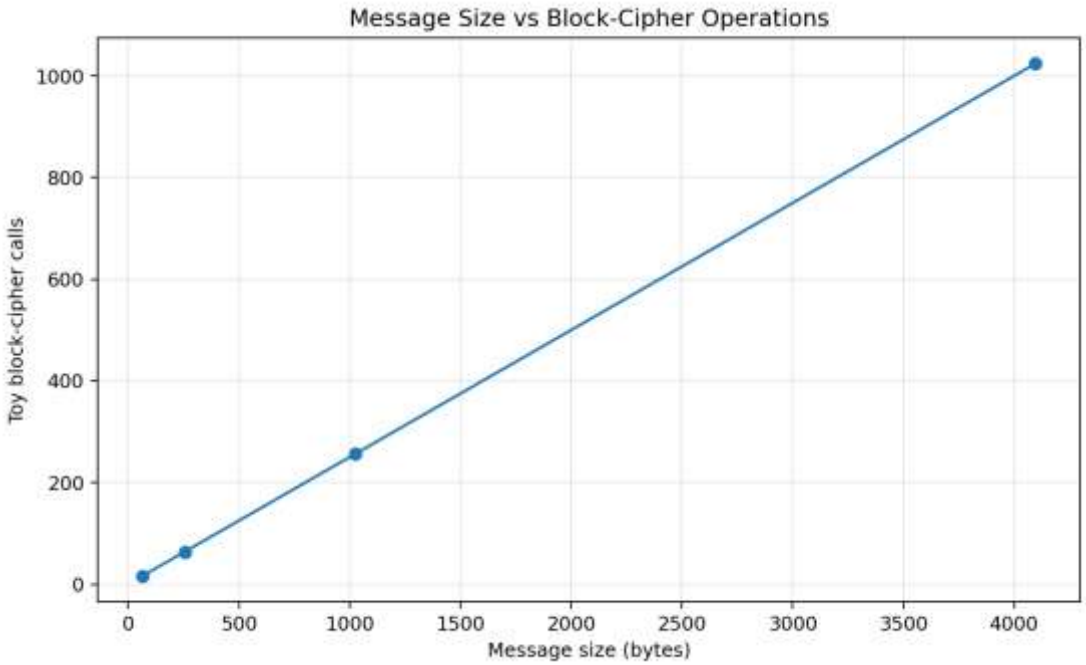


Appendix Figure A7. Performance and integrated comparison execution output.

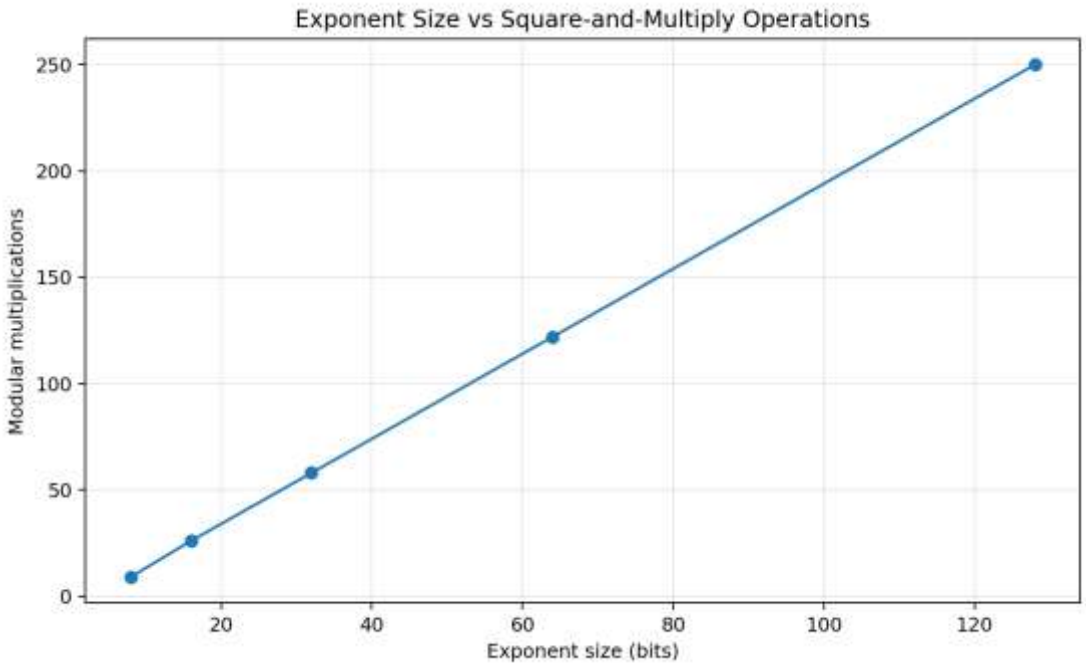
A.5 Experimental Graphs



Appendix Figure A8. Message size versus measured execution time.



Appendix Figure A9. Message size versus toy block-operation count.



Appendix Figure A10. Exponent bit length versus modular multiplication count.

A.6 Raw Experimental Data (CSV extracts)

Table A1. Symmetric performance measurements

message_size_bytes	toy_block_calls	execution_time_microseconds	ciphertext_bytes
64	16	109.89	64
256	64	259.49	256
1024	256	1255.8	1024
4096	1024	4481.0	4096

Table A2. Modular exponentiation measurements

exponent_bits	modular_multiplications	execution_time_microseconds
8	9	3.38
16	26	3.63
32	58	5.25
64	122	9.76
128	250	19.04

Table A3. Primality-test results

candidate	trial_division_prime	fermat_pass	miller_rabin_pass	trial_division_steps
97	True	True	True	8

561	False	True	False	2
1105	False	True	False	4
1729	False	True	False	6